MANUAL

Cómo crear una API REST con ASP.NET Core Web API y Entity Framework Core

Guía paso a paso, sin ambigüedades, para construir el backend que conecta **cualquier frontend** con **cualquier base de datos relacional**.

Versión del manual: 1.1 · octubre 2026
Basado en: el proyecto `P04R01_APIREST` (ASP.NET Core Web API, .NET 8, SQL Server)
Cubre: .NET 8, .NET 9 y .NET 10 (LTS actual) · Visual Studio 2022 / 2026 · CLI `dotnet`
Requiere saber: C# básico y SQL básico

Contenido

1. Antes de empezar 5

1.1 Qué es una API y dónde encaja . . . 5
1.2 Cómo leer este manual . . . 5
1.3 ¿Este manual sirve para cualquier API? . . . 6
1.4 Glosario mínimo . . . 6

2. Paso 0 — Requisitos previos 8

2.1 Software . . . 8
2.2 Qué versión de .NET elegir (octubre 2026) . . . 8
2.3 La base de datos: ¿ya existe o la creará la API? . . . 8

3. Paso 1 — Crear el proyecto con la plantilla correcta 10

3.1 En Visual Studio . . . 10
3.2 Qué crea la plantilla . . . 10
3.3 Equivalente en línea de comandos (VS Code / cualquier editor) . . . 11

4. Paso 2 — Instalar los paquetes NuGet de Entity Framework Core 12

4.1 Qué paquetes y para qué . . . 12
4.2 Tres formas de instalarlos (elige una) . . . 12
4.3 Compilar y recompilar . . . 13
4.4 Si tu base de datos no es SQL Server . . . 13

5. Paso 3 — Crear los modelos (una clase por tabla) 15

5.1 Crear la carpeta Models . . . 15
5.2 ¿Una clase por cada tabla? Sí . . . 15
5.3 Antes de escribir las propiedades: los using . . . 15
5.4 Qué lleva cada clase: una propiedad por columna . . . 16
5.5 Plantilla de modelo . . . 17
5.6 Relaciones entre tablas (llaves foráneas) . . . 17
5.7 Alternativa automática: generar TODOS los modelos desde la BD . . . 18
5.8 Si usas Code First (la API crea las tablas) . . . 19

6. Paso 4 — Crear el DbContext (la conexión con la BD en C#) 20

6.1 Crear la carpeta y la clase . . . 20

6.2 Qué va dentro . . . 20
6.3 ¿Qué es la «clase entidad» y qué es DbSet? . . . 21
6.4 Configuración avanzada (opcional): OnModelCreating . . . 21

7. Paso 5 — Cadena de conexión en appsettings.json 22

7.1 Dónde está . . . 22
7.2 Qué lleva . . . 22
7.3 Formatos de cadena de conexión . . . 22
7.4 Seguridad: no subas contraseñas a git . . . 23

8. Paso 6 — Program.cs: conectar todo y habilitar CORS 24

8.1 Las dos zonas de Program.cs . . . 24
8.2 Registrar el DbContext . . . 24
8.3 CORS: permitir que el frontend consuma la API . . . 25
8.4 Program.cs completo (.NET 8, plantilla con Swashbuckle) . . . 26
8.5 Diferencia en .NET 9 y .NET 10: Swagger . . . 27

9. Paso 7 — Crear los controladores (los endpoints) 29

9.1 Dónde . . . 29
9.2 Crear un controlador vacío . . . 29
9.3 El campo private readonly y el constructor . . . 29
9.4 Los métodos CRUD: plantilla completa . . . 30
9.5 Rutas que quedan y qué responde cada una . . . 32
9.6 Por qué async/await y no la versión síncrona . . . 33

10. Paso 8 — launchSettings.json: cómo se ejecuta la API 34

10.1 Dónde está y para qué sirve . . . 34
10.2 Qué revisar o cambiar . . . 34

11. Paso 9 — Ejecutar y probar la API 36

11.1 Ejecutar . . . 36
11.2 Probar . . . 36
11.3 Solución de problemas frecuentes . . . 36

12. ¿Qué cambia si cambio de base de datos, frontend, versión o herramienta? 38

12.1 Cambiar de base de datos . . . 38
12.2 Cambiar de frontend . . . 38

12.3 Cambiar de versión de .NET 38
12.4 Usar VS Code o la terminal en lugar de Visual Studio 38
12.5 El mismo modelo mental en otros lenguajes 39

13. Observaciones sobre el proyecto P04R01_APIREST **40**

14. Buenas prácticas y siguientes mejoras **41**

15. Lista de verificación final **42**

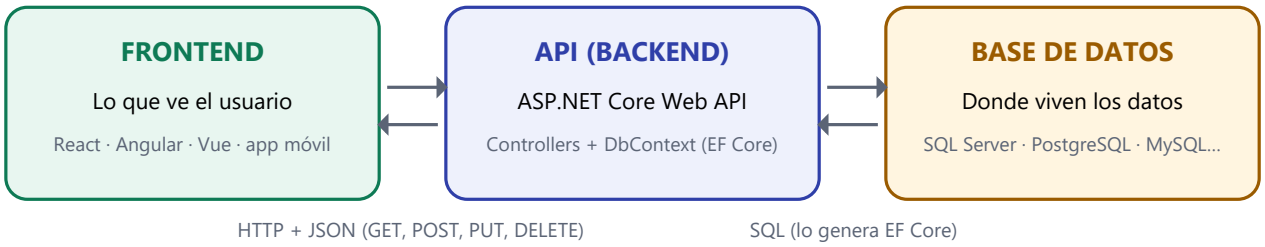
Apéndice A — Ejemplo completo para una base de datos con N tablas **43**

Apéndice B — Comandos de referencia **45**

1. Antes de empezar

1.1 Qué es una API y dónde encaja

Una **API REST** es un programa de **backend** que hace de intermediario entre lo que ve el usuario (el **frontend**) y los datos (la **base de datos**). El frontend nunca habla directo con la base de datos: le pide a la API, por HTTP, que consulte o guarde información, y la API responde en formato **JSON**.



Por eso una misma API puede servir al mismo tiempo a una página en React, a otra en Angular y a una app móvil: a todos les responde el mismo JSON. Lo único que cambia por cada frontend es un permiso llamado **CORS** (Paso 6).

1.2 Cómo leer este manual

Los valores que debes reemplazar con los de tu proyecto aparecen entre comillas angulares «así». Reemplázalos **incluyendo** las comillas angulares « ». Si el marcador está dentro de comillas dobles (`"«NombreConexion»"`), las comillas dobles se quedan. Ejemplo: `«Tabla»` en tu proyecto podría ser `Productos`.

Marcador	Qué significa	Ejemplo (proyecto P04R01_APIREST)
`«Proyecto»`	Nombre del proyecto. También es el *namespace* raíz del código.	`P04R01_APIREST`
`«Tabla»`	Nombre de una tabla de la BD. Se repite por cada tabla: «Tabla1» … «TablaN».	`TBLUsuarios`
`«ClavePrimaria»`	Columna llave primaria de la tabla.	`idUsuarios`
`«Servidor»`	Nombre del servidor de base de datos (y su instancia, si tiene).	`Angelgar` o `Angelgar\SQLEXPRESS`
`«BD»`	Nombre de la base de datos.	`Thona_PT`
`«NombreConexion»`	Nombre que le das a la cadena de conexión en appsettings.json.	`Conexion`
`«Politica»`	Nombre que le das a la política CORS.	`AllowAngular`
`«PuertoFrontend»`	Puerto donde corre el frontend en desarrollo.	`5173` (React/Vite), `4200` (Angular)

Recuadros que verás:

> **Nota:** información adicional que aclara el paso.

> **Buena práctica:** la forma recomendada de hacerlo.

> **Atención:** algo que cambia según la versión o que suele confundir.

> **Error común:** un error frecuente y cómo evitarlo.

1.3 ¿Este manual sirve para cualquier API?

Respuesta corta:

Pregunta	Respuesta
¿Sirve para **cualquier frontend**?	**Sí.** La API responde JSON por HTTP; React, Angular, Vue, apps móviles o Postman la consumen igual. Solo se ajusta el origen en CORS (Paso 6).
¿Sirve para **cualquier base de datos**?	**Sí, para bases relacionales** soportadas por Entity Framework Core: SQL Server, PostgreSQL, MySQL/MariaDB, SQLite, Oracle. Cambian 3 cosas: el paquete NuGet (Paso 2), la cadena de conexión (Paso 5) y el método `UseXxx` (Paso 6).
¿Sirve con **cualquier cantidad de tablas**?	**Sí.** Por cada tabla repites: 1 clase modelo (Paso 3), 1 `DbSet` (Paso 4) y, si la expones, 1 controlador (Paso 7).
¿Es solo para la plantilla de Visual Studio?	Es para **ASP.NET Core Web API con controladores**. Funciona igual desde Visual Studio, VS Code o la línea de comandos (se dan los comandos equivalentes). Si eliges *Minimal APIs* (sin controladores) el Paso 7 cambia.
¿Sigue vigente hoy (2026)?	**Sí.** La estructura es la misma en .NET 8, 9 y 10. Lo que cambió es Swagger en .NET 9+ (Paso 6) y qué versión de Visual Studio necesitas (Paso 0).
¿Sirve para otros lenguajes (Node, Java, Python)?	Los **conceptos** sí (capítulo 12 trae la tabla de equivalencias); el código y los comandos no.
¿Sirve para .NET Framework 4.x (Web API 2 / MVC 5)?	**No.** Es otra tecnología (Web.config, Entity Framework 6). Este manual es para ASP.NET Core (.NET 5 en adelante).

1.4 Glosario mínimo

Término	Qué es, en una línea
Solución (.sln)	Contenedor de Visual Studio que agrupa uno o varios proyectos.
Proyecto (.csproj)	La aplicación en sí (tu API). Las carpetas y clases se agregan **al proyecto**, no a la solución.
Endpoint	Una URL + un verbo HTTP que hace algo. Ej.: `GET /api/Productos`.
Verbos HTTP	`GET` leer · `POST` crear · `PUT` actualizar · `DELETE` eliminar.
JSON	Formato de texto con el que la API recibe y envía datos: `{ "nombre": "Ana" }`.

Término	Qué es, en una línea
ORM	Librería que traduce clases de C# a tablas SQL y viceversa. En .NET es **Entity Framework Core (EF Core)**.
Entidad / Modelo	Clase de C# que representa **una tabla**; cada propiedad es **una columna**.
DbContext	Clase que representa **la conexión a la base de datos** y agrupa todas las tablas.
DbSet<T>	Propiedad del DbContext que representa **una tabla** y permite consultarla/modificarla.
NuGet	El gestor de paquetes (librerías) de .NET. Equivale a npm en JavaScript.
Inyección de dependencias (DI)	ASP.NET crea los objetos que necesitas (como el DbContext) y te los entrega en el constructor.
Middleware	Pasos por los que pasa cada petición HTTP (CORS, HTTPS, autorización…). Se configuran en Program.cs.
CORS	Permiso que la API da a un navegador para que una página de **otro origen** (otro puerto o dominio) la consuma.
Swagger / OpenAPI	Página web generada automáticamente para ver y probar los endpoints de la API.

2. Paso 0 — Requisitos previos

Sin estos requisitos, los demás pasos fallan.

2.1 Software

Herramienta	Para qué	Dónde / cómo
Visual Studio (Community es gratis)	Crear, compilar y ejecutar la API.	Visual Studio Installer → carga de trabajo **«ASP.NET and web development»** (Desarrollo de ASP.NET y web).
SDK de .NET	Compilar y ejecutar. Se instala con Visual Studio.	Verifica en una terminal: `dotnet --list-sdks`
Motor de base de datos	Guardar los datos.	SQL Server Express/Developer, PostgreSQL, MySQL, etc.
Cliente de BD	Crear la BD y ejecutar scripts.	SQL Server Management Studio (SSMS), Azure Data Studio, pgAdmin, MySQL Workbench…
(Opcional) Postman	Probar la API fuera de Swagger.	postman.com

2.2 Qué versión de .NET elegir (octubre 2026)

Versión	Tipo	Soporte hasta	Visual Studio mínimo	EF Core que corresponde
.NET 10	LTS (recomendada)	14-nov-2028	Visual Studio **2026** (v18.0+)	10.0.x
.NET 9	STS	10-nov-2026	Visual Studio 2022 v17.12+	9.0.x
.NET 8	LTS	**10-nov-2026**	Visual Studio 2022 v17.8+	8.0.x

> **Atención:** el proyecto `P04R01_APIREST` usa **.NET 8** con Visual Studio 2022 v17.10. Funciona, pero .NET 8 deja de recibir parches de seguridad el **10 de noviembre de 2026**. Para proyectos nuevos usa **.NET 10** (requiere Visual Studio 2026). Los pasos de este manual son los mismos; las diferencias se marcan con «Atención».

> **Error común:** la versión de los paquetes de EF Core debe tener **el mismo número mayor** que tu .NET (net8.0 → 8.0.x, net10.0 → 10.0.x). En `P04R01_APIREST` el proyecto es net8.0 pero tiene EF Core **6.0.0**: compila, pero es una combinación desactualizada y sin soporte.

2.3 La base de datos: ¿ya existe o la creará la API?

Enfoque	Cuándo usarlo	Cómo se crean los modelos
Database First (la BD ya existe)	Ya tienes las tablas creadas con scripts SQL (como en P04R01).	A mano (Paso 3) o **automáticamente** con `Scaffold-DbContext` (sección 5.7).
Code First (la BD la crea la API)	Empiezas de cero y prefieres definir todo en C#.	Escribes las clases y EF crea las tablas con **migraciones** (sección 5.8).

Este manual sigue **Database First**, que es el caso de P04R01, e indica las diferencias para Code First.

> **Buena práctica:** antes del Paso 1 verifica que puedes conectarte a tu servidor de BD desde SSMS (o el cliente que uses) y anota el **nombre exacto del servidor** que aparece en la ventana de conexión. Lo necesitarás en el Paso 5.

3. Paso 1 — Crear el proyecto con la plantilla correcta

3.1 En Visual Studio

1. Abre Visual Studio → **Crear un proyecto** (Create a new project).
2. En el buscador escribe `web api` y elige **«ASP.NET Core Web API»** con la etiqueta **C#**.
3. Escribe el **nombre del proyecto** («Proyecto») y la ubicación → **Siguiente**.
4. En «Información adicional» configura las opciones de la tabla siguiente → **Crear**.

> **Error común:** hay plantillas con nombres parecidos que **no** sirven: «ASP.NET Core Web App» (páginas Razor), «ASP.NET Core Empty» (vacía) y «ASP.NET Web Application (.NET Framework)» (tecnología antigua).

Opción	Valor recomendado	Por qué
Framework	.NET 10 (o la LTS que soporte tu Visual Studio)	Ver Paso 0.
Authentication type	None	La autenticación (JWT, etc.) se agrega después.
Configure for HTTPS	Marcado	Crea el perfil https; puedes seguir usando el perfil http en desarrollo.
Enable container support (Docker)	Desmarcado	No es necesario para empezar.
Enable OpenAPI support	Marcado	Genera la documentación de la API (Swagger/OpenAPI).
Use controllers	**Marcado (obligatorio para este manual)**	Si lo desmarcas se crea una *Minimal API* sin carpeta Controllers y el Paso 7 cambia.
Do not use top-level statements	Desmarcado	Program.cs queda corto, como en los ejemplos de este manual.

3.2 Qué crea la plantilla

Archivo / carpeta	Para qué sirve	¿Lo modificas?
`«Proyecto».sln`	La solución (contenedor).	No.
`«Proyecto».csproj`	Configuración del proyecto: versión de .NET y **paquetes NuGet**.	Lo modifica NuGet por ti (Paso 2).
`Program.cs`	Punto de arranque: registra servicios y middleware.	**Sí** (Paso 6).
`appsettings.json`	Configuración: cadena de conexión, logging…	**Sí** (Paso 5).
`appsettings.Development.json`	Configuración que solo aplica en desarrollo.	Opcional.
`Properties/launchSettings.json`	Cómo se ejecuta al presionar Iniciar: puertos, perfil, página inicial.	Solo si quieres (Paso 8).

Archivo / carpeta	Para qué sirve	¿Lo modificas?
`Controllers/` + `WeatherForecastController.cs`	Carpeta de controladores con un **ejemplo**.	La carpeta se usa (Paso 7); el ejemplo se puede borrar.
`WeatherForecast.cs`	Clase del ejemplo.	Se puede borrar junto con su controlador.
`«Proyecto».http`	Archivo para probar endpoints desde Visual Studio.	Opcional (Paso 9).

> **Nota:** la carpeta **Controllers ya existe**: la crea la plantilla. No hace falta crearla en el Paso 7 (en P04R01 se ve con `WeatherForecastController.cs`).

3.3 Equivalente en línea de comandos (VS Code / cualquier editor)

Terminal

```
dotnet new webapi --use-controllers -n «Proyecto»
cd «Proyecto»
dotnet run
```

> **Atención:** desde .NET 8, `dotnet new webapi` **sin** `--use-controllers` crea una Minimal API. Agrega siempre esa opción.

4. Paso 2 — Instalar los paquetes NuGet de Entity Framework Core

Instala estos paquetes **antes** de crear los modelos y el DbContext: sin ellos, `DbContext`, `DbSet` y `UseSqlServer` no existen y el código no compila.

4.1 Qué paquetes y para qué

Paquete	Obligatorio	Para qué
`Microsoft.EntityFrameworkCore.SqlServer`	Sí (para SQL Server)	El **proveedor**: sabe hablar con SQL Server. Incluye EF Core. Para otra BD se usa otro proveedor (4.4).
`Microsoft.EntityFrameworkCore.Tools`	Recomendado	Comandos de la **Consola del Administrador de paquetes**: `Scaffold-DbContext`, `Add-Migration`, `Update-Database`.
`Microsoft.EntityFrameworkCore.Design`	Solo si usas la CLI `dotnet ef`	Necesario para los comandos `dotnet ef …` (ya viene incluido con Tools).

> **Atención:** instala **la misma versión** en todos los paquetes de EF Core, y que coincida con tu .NET: net8.0 → 8.0.x, net9.0 → 9.0.x, net10.0 → 10.0.x.

4.2 Tres formas de instalarlos (elige una)

Forma A — Interfaz gráfica (la más fácil)

1. En el **Explorador de soluciones**, clic derecho sobre el **proyecto** (no sobre la solución) → **Administrar paquetes NuGet…** (Manage NuGet Packages).
2. Pestaña **Examinar** (Browse) → busca `Microsoft.EntityFrameworkCore.SqlServer`.
3. Selecciónalo, elige en **Versión** la que corresponde a tu .NET (ej. la última 8.0.x para net8.0) → **Instalar** → acepta la licencia.
4. Repite con `Microsoft.EntityFrameworkCore.Tools` **con la misma versión**.
5. Revisa la pestaña **Instalado** (Installed): ambos deben aparecer.

Forma B — Consola del Administrador de paquetes (Package Manager Console)

Menú **Herramientas** → **Administrador de paquetes NuGet** → **Consola del Administrador de paquetes**. Se abre abajo una consola `PM>`.

> **Error común:** antes de escribir los comandos, en la parte de arriba de la consola cambia la lista **«Proyecto predeterminado»** al proyecto de tu API. Si la solución tiene otros proyectos (Web Forms, WinForms, bibliotecas de clases), la consola instala en el que esté seleccionado, y si es de .NET Framework la instalación falla («el paquete no contiene referencias de ensamblado o archivos de contenido compatibles con dicho marco»). Otra forma: agrega `-ProjectName «Proyecto»` al final de cada comando.

Escribe los comandos con la versión **exacta**, la misma en ambos paquetes:

Consola del Administrador de paquetes (PM>) — para .NET 10 usa una versión 10.0.x exacta

```
Install-Package Microsoft.EntityFrameworkCore.SqlServer -Version 8.0.31 -ProjectName «Proyecto»
Install-Package Microsoft.EntityFrameworkCore.Tools -Version 8.0.31 -ProjectName «Proyecto»
```

> **Atención:** la consola **no acepta comodines**: con `-Version 8.0.*` responde «Error al analizar la entrada del parámetro de versión 8.0.* en una versión semántica válida». La versión exacta más reciente se consulta en nuget.org (pestaña *Versions* del paquete) o en la lista **Versión** de la Forma A. El comodín `8.0.*` solo funciona en la terminal (Forma C, `dotnet add package`).

Forma C — Terminal (CLI), sirve también en VS Code

Terminal, dentro de la carpeta del proyecto

```
dotnet add package Microsoft.EntityFrameworkCore.SqlServer --version 8.0.*
dotnet add package Microsoft.EntityFrameworkCore.Design --version 8.0.*

# Solo una vez por computadora, para tener el comando "dotnet ef":
dotnet tool install --global dotnet-ef
```

4.3 Compilar y recompilar

Acción	Dónde	Atajo	Qué hace
Compilar solución (Build)	Menú **Compilar** → **Compilar solución**	`Ctrl+Shift+B`	Compila solo lo que cambió. Úsalo después de cada paso.
Recompilar solución (Rebuild)	Menú **Compilar** → **Recompilar solución**	—	Borra lo compilado y compila todo de nuevo. Úsalo si hay errores raros después de instalar o actualizar paquetes.
Equivalente CLI	Terminal	`dotnet build`	Igual que Compilar.

El resultado debe decir **«Compilación correcta» / «Build succeeded»** con 0 errores en la ventana **Lista de errores** (Ver → Lista de errores).

4.4 Si tu base de datos no es SQL Server

Base de datos	Paquete NuGet (proveedor)	Método en Program.cs (Paso 6)
SQL Server	`Microsoft.EntityFrameworkCore.SqlServer`	`UseSqlServer(cadena)`
PostgreSQL	`Npgsql.EntityFrameworkCore.PostgreSQL`	`UseNpgsql(cadena)`
MySQL / MariaDB	`Pomelo.EntityFrameworkCore.MySql`	`UseMySql(cadena, ServerVersion.AutoDetect(cadena))`
SQLite	`Microsoft.EntityFrameworkCore.Sqlite`	`UseSqlite(cadena)`
Oracle	`Oracle.EntityFrameworkCore`	`UseOracle(cadena)`

> **Atención:** los proveedores de terceros (Npgsql, Pomelo, Oracle) publican su versión para cada .NET un poco después. Antes de instalar, revisa en la página del paquete en nuget.org que exista una versión para tu .NET.

5. Paso 3 — Crear los modelos (una clase por tabla)

5.1 Crear la carpeta Models

1. En el **Explorador de soluciones**, clic derecho sobre el **proyecto** de la API: el nodo con el nombre de tu proyecto, debajo de la solución. En un proyecto ASP.NET Core Web API su ícono es un **globo azul** (proyecto web). Luego **Agregar** → **Nueva carpeta**.
2. Nómbrala `Models`.

> **Error común:** las carpetas se agregan al **proyecto**, no a la solución. Si las agregas a la solución, se crean «carpetas de solución», que son virtuales y no compilan código.

> **Nota:** `Models` es el nombre convencional en inglés (también `Entities`). Puedes llamarla `Modelos`, pero el *namespace* de las clases será `«Proyecto».Modelos` y deberás usar ese nombre en todos los `using`. Elige uno y mantenlo en todo el proyecto.

5.2 ¿Una clase por cada tabla? Sí

Cada tabla de la base de datos que la API vaya a usar = una clase en Models. Si tienes 10 tablas, tendrás 10 clases:

```
Models/
├── «Tabla1».cs      → representa la tabla «Tabla1»
├── «Tabla2».cs      → representa la tabla «Tabla2»
├── ...
└── «TablaN».cs      → representa la tabla «TablaN»
```

Para crear cada una: clic derecho en la carpeta **Models** → **Agregar** → **Clase...** → nombre `«Tabla».cs` → **Agregar**.

> **Buena práctica:** si tienes muchas tablas, no las escribas a mano: genera todas las clases de un solo comando con `Scaffold-DbContext` (sección 5.7).

5.3 Antes de escribir las propiedades: los using

Al crear la clase (Agregar → Clase...), Visual Studio genera solo el `namespace` y la clase vacía: **no** agrega los `using` de las anotaciones. Escríbelos tú, arriba del `namespace`:

Models/«Tabla».cs — primeras líneas

```
using System.ComponentModel.DataAnnotations;          // [Key], [Required], [StringLength]
using System.ComponentModel.DataAnnotations.Schema;  // [Table], [Column],
                                                      // [ForeignKey], [DatabaseGenerated]
```

using	Para qué	¿Cuándo hace falta?
`System.ComponentModel.DataAnnotations`	Anotaciones de **validación y llave**: `[Key]`, `[Required]`, `[StringLength]`, `[Range]`.	Casi siempre (todas las tablas llevan `[Key]` si la llave no se llama `Id`).

using	Para qué	¿Cuándo hace falta?
`System.ComponentModel.DataAnnotations.Schema`	Anotaciones de **mapeo a la BD**: `[Table]`, `[Column]`, `[ForeignKey]`, `[DatabaseGenerated]`.	Cuando usas alguna de esas anotaciones.

> **Atajo:** si escribes `[Key]` sin el `using`, aparece subrayado en rojo. Pon el cursor encima y presiona **Ctrl+.** (control + punto): Visual Studio ofrece agregar el `using` correcto. Los tipos básicos (`DateTime`, `List<>`) no necesitan `using`, porque la plantilla trae *ImplicitUsings* activado.

5.4 Qué lleva cada clase: una propiedad por columna

Tipo en SQL Server	Tipo en C#	Si la columna acepta NULL
INT	`int`	`int?`
BIGINT / SMALLINT / TINYINT	`long` / `short` / `byte`	`long?` / `short?` / `byte?`
BIT	`bool`	`bool?`
DECIMAL, NUMERIC, MONEY	`decimal`	`decimal?`
FLOAT / REAL	`double` / `float`	`double?` / `float?`
NVARCHAR, VARCHAR, CHAR, TEXT	`string`	`string?`
DATETIME, DATETIME2, DATE	`DateTime` (DATE también puede ser `DateOnly`)	`DateTime?`
TIME	`TimeSpan` (o `TimeOnly`)	`TimeSpan?`
UNIQUEIDENTIFIER	`Guid`	`Guid?`
VARBINARY	`byte[]`	`byte[]?`

Anotaciones (atributos) más usadas

Atributo	Para qué	¿Cuándo es obligatorio?
`[Key]`	Marca la llave primaria.	Cuando la columna **no** se llama `Id` ni `«Clase»Id`. En P04R01 es obligatorio porque la llave es `idUsuarios`.
`[Table("nombre")]`	Nombre real de la tabla en la BD.	Cuando la clase se llama distinto a la tabla.
`[Column("nombre")]`	Nombre real de la columna.	Cuando la propiedad se llama distinto a la columna.
`[Required]`	No acepta vacío; la API responde 400 si falta.	Opcional (validación).
`[StringLength(100)]`	Largo máximo del texto.	Opcional (validación); usa el mismo largo que en la BD.
`[DatabaseGenerated(...)]`	Valor que pone la BD (identity, DEFAULT, calculado).	Para columnas con DEFAULT que no envía la API.

Recuerda: estos atributos necesitan los `using` de la sección 5.3.

5.5 Plantilla de modelo

Models/«Tabla».cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace «Proyecto».Models
{
    [Table("«Tabla»")]                // solo si la clase se llama distinto a la tabla
    public class «Tabla»
    {
        [Key]                         // llave primaria (si no se llama Id / «Tabla»Id)
        public int «ClavePrimaria» { get; set; }

        [Required, StringLength(100)]  // NVARCHAR(100) NOT NULL
        public string Nombre { get; set; } = string.Empty;

        [StringLength(150)]
        public string? Descripcion { get; set; }   // NVARCHAR(150) NULL  -> string?

        public DateTime FechaAlta { get; set; }    // DATETIME NOT NULL
        public decimal Precio { get; set; }        // DECIMAL(10,2)
        public bool Activo { get; set; }           // BIT
    }
}
```

Ejemplo real de P04R01 (`Models/TBLUsuarios.cs`), tabla `TBLUsuarios(idUsuarios INT IDENTITY PK, nombre NVARCHAR(100) NOT NULL, hobby NVARCHAR(150) NULL, fechanacimiento DATETIME NOT NULL, email NVARCHAR(100) NOT NULL)`:

Models/TBLUsuarios.cs (simplificado)

```
public class TBLUsuarios
{
    [Key]
    public int idUsuarios { get; set; }
    public string nombre { get; set; }
    public string? hobby { get; set; }
    public DateTime fechanacimiento { get; set; }
    public string email { get; set; }
}
```

> **Atención:** con `<Nullable>enable</Nullable>` (activado por la plantilla), una propiedad `string` sin `?` se trata como **obligatoria**: si el JSON no la trae, la API responde 400 automáticamente. Usa `string?` para columnas que aceptan NULL.

5.6 Relaciones entre tablas (llaves foráneas)

- Para guardar y leer registros **basta con la columna FK** como `int` (por ejemplo, `public int IdChofer { get; set; }`). La base de datos ya valida la llave foránea; EF no necesita más.
- Las **propiedades de navegación** (`Chofer`, `List<Rutas>`) son **opcionales**. Sirven para traer los datos del registro relacionado en la misma consulta con `.Include(...)`.

- **No pongas `[Key]` en una llave foránea.** `[Key]` es solo para la llave primaria de la tabla.
- En C# el orden de las clases no importa: una clase puede usar otra que creas después. El subrayado rojo desaparece cuando las dos existen.
- Recomendación para empezar: navegación **solo en la tabla hija** (o ninguna). Con navegación en los dos sentidos aparece el error de ciclo JSON (ver el Error común de esta sección).

Ejemplo completo, con la columna FK y navegación en ambos sentidos:

Relación uno a muchos

```
public class «TablaHija»
{
    [Key]
    public int Id«TablaHija» { get; set; }

    public int Id«TablaPadre» { get; set; }          // columna FK
    [ForeignKey(nameof(Id«TablaPadre»))]
    public «TablaPadre»? «TablaPadre» { get; set; }   // navegación: el registro padre
}

public class «TablaPadre»
{
    [Key]
    public int Id«TablaPadre» { get; set; }
    public List<«TablaHija»> «TablaHija»s { get; set; } = new();   // navegación: sus hijos
}
```

Error común: al devolver entidades con navegación en ambos sentidos, la API puede fallar con «A possible object cycle was detected». Solución recomendada: devolver DTOs (capítulo 14). Solución rápida: en Program.cs, `AddControllers().AddJsonOptions(o => o.JsonSerializerOptions.ReferenceHandler = ReferenceHandler.IgnoreCycles)`.

5.7 Alternativa automática: generar TODOS los modelos desde la BD

Con la base de datos ya creada, un solo comando genera las N clases de modelo **y** el DbContext (Pasos 3 y 4 juntos). Requiere el paquete Tools (Paso 2).

Consola del Administrador de paquetes (PM>) — haz primero el Paso 5

```
Scaffold-DbContext "Name=ConnectionStrings:«NombreConexion»" Microsoft.EntityFrameworkCore.SqlServer `
    -OutputDir Models -ContextDir Context -Context AppDbContext -DataAnnotations

# Solo algunas tablas:     -Tables «Tabla1»,«Tabla2»
# Regenerar (sobrescribe): -Force
```

Equivalente en terminal (CLI)

```
dotnet ef dbcontext scaffold "Name=ConnectionStrings:«NombreConexion»" Microsoft.EntityFrameworkCore.SqlServer \
    -o Models --context-dir Context -c AppDbContext --data-annotations
```

> **Atención:** usar `Name=ConnectionStrings:«NombreConexion»` hace que el código generado lea la cadena desde appsettings.json. Si en su lugar pegas la cadena completa, el comando la escribe dentro de `OnConfiguring` del DbContext, con tu contraseña visible: en ese caso **borra el método** `OnConfiguring` y deja la configuración en Program.cs (Paso 6).

5.8 Si usas Code First (la API crea las tablas)

Escribes las clases (5.5), el DbContext (Paso 4), la conexión (Paso 5) y Program.cs (Paso 6), y después ejecutas:

Consola del Administrador de paquetes

```
Add-Migration Inicial      # genera el script de creación en la carpeta Migrations
Update-Database            # lo ejecuta contra la BD (la crea si no existe)

# CLI: dotnet ef migrations add Inicial   y   dotnet ef database update
```

Cada vez que cambies un modelo: `Add-Migration «NombreDelCambio»` y `Update-Database`.

6. Paso 4 — Crear el DbContext (la conexión con la BD en C#)

6.1 Crear la carpeta y la clase

1. Clic derecho en el **proyecto** → **Agregar** → **Nueva carpeta** → `Context` (también se usa `Data`).
2. Clic derecho en la carpeta **Context** → **Agregar** → **Clase...** → `AppDbContext.cs`.

`AppDbContext` es un nombre **convencional**, no obligatorio: puedes llamarlo `TiendaDbContext`, por ejemplo, siempre que uses el mismo nombre en Program.cs y en los controladores.

6.2 Qué va dentro

En este archivo se importan las librerías, se hereda de `DbContext`, se crea el constructor y se declara un `DbSet` por tabla: `Context/AppDbContext.cs`.

Context/AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;   // (1) DbContext, DbSet, DbContextOptions
using «Proyecto».Models;               // (2) tus clases de modelo

namespace «Proyecto».Context
{
    public class AppDbContext : DbContext                       // (3) HEREDA de DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options)  // (4) constructor
            : base(options)
        {
        }

        // (5) Un DbSet por cada tabla:  DbSet<ClaseModelo> NombreDeLaTabla
        public DbSet<«Tabla1»> «Tabla1» { get; set; }
        public DbSet<«Tabla2»> «Tabla2» { get; set; }
        // ... una línea por tabla ...
        public DbSet<«TablaN»> «TablaN» { get; set; }
    }
}
```

#	Línea	Qué hace y por qué
1	`using Microsoft.EntityFrameworkCore;`	Trae `DbContext`, `DbSet` y `DbContextOptions`. Viene del paquete del Paso 2: sin él, esta línea marca error.
2	`using «Proyecto».Models;`	Permite usar tus clases de modelo (`«Tabla1»`...) sin escribir el namespace completo.
3	`: DbContext`	**AppDbContext hereda de DbContext** (no al revés). Así obtiene toda la funcionalidad de EF Core: consultar, guardar, transacciones.
4	Constructor con `DbContextOptions<AppDbContext>`	Recibe la configuración (qué BD y qué cadena de conexión) que se define en Program.cs (Paso 6) y se la pasa a la clase base con `: base(options)`. El cuerpo va vacío.
5	`DbSet<«Tabla»> «Tabla»`	Representa **una tabla**. Por convención, EF usa el **nombre de la propiedad** como nombre de la tabla en la BD.

> **Error común:** la clase `AppDbContext` **hereda de** `DbContext` (no al revés), y además tiene un **constructor** que recibe las opciones.

6.3 ¿Qué es la «clase entidad» y qué es DbSet?

- **Clase entidad** = la clase de modelo que creaste en el Paso 3 (por ejemplo `TBLUsuarios`). «Entidad» y «modelo» se usan como sinónimos.
- **DbSet<T>** = la «tabla» dentro de C#. Con ella haces todo:

Código C#	SQL equivalente (lo genera EF Core)
`await _context.«Tabla».ToListAsync()`	`SELECT * FROM «Tabla»`
`await _context.«Tabla».FindAsync(5)`	`SELECT … WHERE «ClavePrimaria» = 5`
`_context.«Tabla».Where(x => x.Activo).ToListAsync()`	`SELECT … WHERE Activo = 1`
`_context.«Tabla».Add(obj)` + `SaveChangesAsync()`	`INSERT INTO «Tabla» …`
modificar propiedades + `SaveChangesAsync()`	`UPDATE «Tabla» SET …`
`_context.«Tabla».Remove(obj)` + `SaveChangesAsync()`	`DELETE FROM «Tabla» WHERE …`

Ejemplo real de P04R01: `public DbSet<TBLUsuarios> TBLUsuarios { get; set; }`. La clase es `TBLUsuarios` y la tabla en SQL también se llama `TBLUsuarios`, por eso funciona sin `[Table]`.

6.4 Configuración avanzada (opcional): OnModelCreating

Lo que no se puede (o no conviene) expresar con atributos se configura sobrescribiendo `OnModelCreating` dentro de AppDbContext:

Context/AppDbContext.cs — dentro de la clase

```
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<«Tabla»>().ToTable("«nombre_real_en_bd»");          // nombre de tabla
    modelBuilder.Entity<«TablaDetalle»>().HasKey(d => new { d.IdPedido, d.IdProducto }); // llave compuesta
    modelBuilder.Entity<«Tabla»>().Property(t => t.Precio).HasPrecision(10, 2);   // DECIMAL(10,2)
    modelBuilder.Entity<«Tabla»>().HasIndex(t => t.Email).IsUnique();            // índice único
}
```

7. Paso 5 — Cadena de conexión en appsettings.json

7.1 Dónde está

En el **Explorador de soluciones**, en la **raíz del proyecto** (al mismo nivel que Program.cs), está `appsettings.json`. Si tiene una flecha a la izquierda, al expandirla aparece `appsettings.Development.json`.

7.2 Qué lleva

Agrega la sección `ConnectionStrings` (con ese nombre exacto, en plural). Dentro, una o más conexiones con el nombre que quieras («NombreConexion»):

appsettings.json

```
{
  "Logging": {
    "LogLevel": { "Default": "Information", "Microsoft.AspNetCore": "Warning" }
  },
  "AllowedHosts": "*",

  "ConnectionStrings": {
    "«NombreConexion»": "Server=«Servidor»;Database=«BD»;Integrated Security=True;TrustServerCertificate=True;"
  }
}
```

Error común: JSON es estricto: comillas dobles, una **coma entre secciones** y **ninguna coma después del último elemento**. Una coma de más o de menos impide que la API arranque.

Error común: dentro de la cadena de conexión se usa `=`, no `:`. El `:` solo separa el nombre de la conexión de su valor en el JSON. Escribir `Server:«Servidor»;Database:«BD»;…` compila, pero la API falla al conectarse con «Format of the initialization string does not conform to specification starting at index 0».

Parte	Separador
`"«NombreConexion»"` → su valor	`:` (es JSON)
`Server` → `«Servidor»` (dentro de la cadena)	`=`
Entre opciones (`Server=…;Database=…`)	`;`

Atención: el nombre «NombreConexion» debe ser **idéntico** al que usarás en Program.cs con `GetConnectionString("«NombreConexion»")`. En P04R01 se llama `Conexion`.

7.3 Formatos de cadena de conexión

Caso	Cadena
SQL Server, autenticación de Windows	`Server=«Servidor»;Database=«BD»;Integrated Security=True;TrustServerCertificate=True;`

Caso	Cadena
SQL Server, usuario y contraseña	`Server=«Servidor»;Database=«BD»;User Id=«usuario»;Password=«clave»;TrustServerCertificate=True;`
SQL Server Express (instancia con nombre)	`Server=«Servidor»\\SQLEXPRESS;Database=«BD»;Integrated Security=True;TrustServerCertificate=True;` (en JSON la `\` se escribe doble)
SQL Server LocalDB	`Server=(localdb)\\MSSQLLocalDB;Database=«BD»;Trusted_Connection=True;`
PostgreSQL	`Host=localhost;Port=5432;Database=«BD»;Username=«usuario»;Password=«clave»`
MySQL / MariaDB	`Server=localhost;Port=3306;Database=«BD»;User=«usuario»;Password=«clave»;`
SQLite	`Data Source=«archivo».db`

- `TrustServerCertificate=True`: necesario en desarrollo con SQL Server desde EF Core 7, porque el certificado local no es de confianza. Sin esto aparece el error «The certificate chain was issued by an authority that is not trusted».
- `Integrated Security=True` y `Trusted_Connection=True` son **la misma opción**. Usa solo una (en P04R01 aparecían ambas con valores opuestos).
- `MultipleActiveResultSets=True` (opcional): permite varias consultas abiertas a la vez en la misma conexión.

7.4 Seguridad: no subas contraseñas a git

Si la cadena lleva usuario y contraseña, guárdala en **User Secrets** en lugar de appsettings.json: clic derecho en el proyecto → **Administrar secretos de usuario** (Manage User Secrets) → se abre `secrets.json`, donde pegas la misma sección `ConnectionStrings`. ASP.NET la lee igual y el archivo **no** queda dentro del proyecto.

8. Paso 6 — Program.cs: conectar todo y habilitar CORS

8.1 Las dos zonas de Program.cs

Program.cs se divide en dos partes separadas por la línea `var app = builder.Build();`. **El orden importa:**

Zona	Va	Qué se pone ahí	Se escribe como
1. Servicios	**Antes** de `builder.Build()`	Lo que la app necesita tener disponible: DbContext, CORS, controladores, Swagger.	`builder.Services.AddXxx(...)`
2. Pipeline (middleware)	**Después** de `builder.Build()`	Por dónde pasa cada petición, en orden: Swagger, CORS, HTTPS, autorización, controladores.	`app.UseXxx()` / `app.MapXxx()`

8.2 Registrar el DbContext

Arriba del archivo agrega los `using`:

Program.cs — primeras líneas

```
using Microsoft.EntityFrameworkCore;   // trae UseSqlServer
using «Proyecto».Context;              // trae AppDbContext
```

Y en la zona de servicios (después de `var builder = WebApplication.CreateBuilder(args);`):

Program.cs — zona de servicios

```
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("«NombreConexion»")));
```

Parte	Qué significa
`builder.Services.AddDbContext<AppDbContext>`	Registra AppDbContext en el contenedor de inyección de dependencias: a partir de aquí, ASP.NET puede **crearlo y entregarlo** a cualquier controlador que lo pida. Crea uno nuevo por cada petición HTTP.
`options.UseSqlServer(...)`	Le dice a EF Core **qué motor de BD** usar. Para otra BD cambia este método (tabla 4.4).
`builder.Configuration.GetConnectionString("«NombreConexion»")`	Lee la cadena del Paso 5, en `appsettings.json → ConnectionStrings → «NombreConexion»`.

Esto es exactamente lo que recibe el constructor de AppDbContext (`DbContextOptions<AppDbContext> options`) del Paso 4.

> **Error común:** el nombre de la conexión es un **texto**, así que va entre comillas dobles y se escribe igual que en appsettings.json (respetando mayúsculas). Sin comillas, C# cree que es una variable y marca el error **CS0103: El nombre 'conexion' no existe en el contexto actual**. Al reemplazar un marcador «...» se quitan solo las comillas angulares « », **no** las comillas dobles `"  "` que lo rodean.

Ejemplo con la conexión llamada "conexion"

```
options.UseSqlServer(builder.Configuration.GetConnectionString(conexion));    // INCORRECTO: CS0103
options.UseSqlServer(builder.Configuration.GetConnectionString("conexion")); // CORRECTO
```

8.3 CORS: permitir que el frontend consuma la API

¿Qué es CORS? *Cross-Origin Resource Sharing* (intercambio de recursos entre orígenes) es una regla de seguridad **de los navegadores**.

- Un **origen** es la combinación **protocolo + dominio + puerto**. `http://localhost:5173` y `http://localhost:5290` son orígenes **distintos**, porque cambia el puerto, aunque los dos estén en tu computadora.
- Cuando una página web (por ejemplo React en `http://localhost:5173`) pide datos con `fetch` a una API que está en **otro** origen (`http://localhost:5290`), el navegador primero pregunta a la API si permite a ese origen. Si la API no responde que sí, el navegador **bloquea la respuesta** y en la consola aparece `blocked by CORS policy`.
- **CORS no lo aplica la API, lo aplica el navegador.** La API solo dice qué orígenes acepta. Por eso Swagger, Postman, una app móvil, WinForms o un servidor (por ejemplo, Web Forms usando `HttpClient` desde el code-behind) **no necesitan CORS**: no son páginas ejecutándose en un navegador.
- Se configura en **dos lugares** de Program.cs, con el **mismo nombre de política**:

Dónde	Código	Para qué
Zona 1 (servicios), antes de `builder.Build()`	`builder.Services.AddCors(...)` + `AddPolicy("«Politica»", ...)`	**Define** la política: qué orígenes, encabezados y métodos se permiten.
Zona 2 (pipeline), después de `builder.Build()`	`app.UseCors("«Politica»");`	**Aplica** la política a cada petición. Va antes de `UseAuthorization` y `MapControllers`.

Método de la política	Qué permite
`WithOrigins("http://localhost:«PuertoFrontend»")`	Solo esos orígenes, escritos exactos y sin `/` final.
`AllowAnyHeader()`	Cualquier encabezado (`Content-Type`, `Authorization`...).
`AllowAnyMethod()`	Cualquier verbo HTTP (GET, POST, PUT, DELETE). Sin esto, GET funciona y PUT o DELETE se bloquean.

El código completo de los dos lugares:

Program.cs

```
// ZONA 1 — servicios (antes de builder.Build())
builder.Services.AddCors(options =>
{
    options.AddPolicy("«Politica»", policy =>
    {
        policy.WithOrigins("http://localhost:«PuertoFrontend»")   // uno o varios, separados por coma
              .AllowAnyHeader()                                   // permite Content-Type, Authorization...
              .AllowAnyMethod();                                  // permite GET, POST, PUT, DELETE
    });
});

// ZONA 2 — pipeline (después de builder.Build()), ANTES de UseAuthorization y MapControllers
app.UseCors("«Politica»");
```

¿Cambia algo entre React y Angular?

No. Los pasos son idénticos; solo cambia la URL de `WithOrigins`, que es donde corre el frontend en desarrollo:

Frontend	Origen típico en desarrollo	Cómo saberlo con certeza
React con Vite	`http://localhost:5173`	Lo imprime la terminal al ejecutar `npm run dev`.
React con Create React App (antiguo) / Next.js	`http://localhost:3000`	Terminal al ejecutar `npm start` / `npm run dev`.
Angular	`http://localhost:4200`	Terminal al ejecutar `ng serve`.
Vue con Vite	`http://localhost:5173`	Terminal al ejecutar `npm run dev`.
Producción	`https://«tu-dominio».com`	El dominio donde publiques el frontend.
App móvil nativa, Postman, otro backend	No necesitan CORS	CORS solo lo aplica el **navegador**.

Para permitir React y Angular a la vez: `policy.WithOrigins("http://localhost:5173", "http://localhost:4200")`.

> **Error común:** errores típicos de CORS: (1) escribir `.AllowAnyHeader()` dos veces y olvidar `.AllowAnyMethod()`; entonces GET funciona pero PUT y DELETE se bloquean (pasó en P04R01). (2) Poner `app.UseCors` después de `app.MapControllers()`. (3) Que el nombre de `AddPolicy` y el de `UseCors` no coincidan. (4) Poner una `/` al final del origen (`http://localhost:5173/` **no** coincide).

8.4 Program.cs completo (.NET 8, plantilla con Swashbuckle)

Program.cs

```
using Microsoft.EntityFrameworkCore;
using «Proyecto».Context;

var builder = WebApplication.CreateBuilder(args);

// ==================== ZONA 1: SERVICIOS ====================
// 1) Base de datos (Paso 5 + Paso 4)
builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("«NombreConexion»")));

// 2) CORS para el/los frontend(s)
builder.Services.AddCors(options =>
{
    options.AddPolicy("«Politica»", policy =>
        policy.WithOrigins("http://localhost:5173", "http://localhost:4200")
              .AllowAnyHeader()
              .AllowAnyMethod());
});

// 3) Controladores y documentación (ya venían en la plantilla)
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// ==================== ZONA 2: PIPELINE ====================
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();          // página en /swagger
}

app.UseCors("«Politica»");      // antes de Authorization y MapControllers
app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();            // activa las rutas de la carpeta Controllers

app.Run();
```

8.5 Diferencia en .NET 9 y .NET 10: Swagger

> **Atención:** desde .NET 9 la plantilla **ya no incluye Swashbuckle** (Swagger UI). En su lugar trae `Microsoft.AspNetCore.OpenApi`, que genera el documento en `/openapi/v1.json` pero **sin página visual**. Las líneas `AddEndpointsApiExplorer`, `AddSwaggerGen`, `UseSwagger` y `UseSwaggerUI` se reemplazan por las de abajo. Todo lo demás (DbContext, CORS, controladores) es igual.

Program.cs en .NET 9 / .NET 10

```
// Zona 1 (servicios) — lo pone la plantilla:
builder.Services.AddControllers();
builder.Services.AddOpenApi();

// Zona 2 (pipeline):
if (app.Environment.IsDevelopment())
{
    app.MapOpenApi();                                          // /openapi/v1.json

    // Para tener una página visual elige UNA opción:
    // a) Instala el paquete Swashbuckle.AspNetCore y agrega:
    app.UseSwaggerUI(o => o.SwaggerEndpoint("/openapi/v1.json", "v1"));   // /swagger
    // b) O instala Scalar.AspNetCore y agrega:  app.MapScalarApiReference();  // /scalar/v1
}
```

> **Nota:** no mezcles `AddOpenApi()` con `AddSwaggerGen()`: usa la generación nativa (`AddOpenApi`) y solo la **interfaz** de Swashbuckle o Scalar.

9. Paso 7 — Crear los controladores (los endpoints)

9.1 Dónde

En la carpeta `Controllers`, que **ya existe** porque la creó la plantilla (Paso 1). Si por algún motivo no existe: clic derecho en el proyecto → Agregar → Nueva carpeta → `Controllers`.

> **Nota:** se crea **un controlador por cada tabla** que quieras exponer: `«Tabla1»Controller`, `«Tabla2»Controller` … `«TablaN»Controller`. No todas las tablas necesitan controlador (por ejemplo, una tabla intermedia de una relación muchos a muchos).

9.2 Crear un controlador vacío

1. Clic derecho en la carpeta **Controllers** → **Agregar** → **Controlador…** (Add → Controller).
2. A la izquierda elige **API**; al centro, **«Controlador de API: en blanco»** (API Controller - Empty) → **Agregar**.
3. Nombre: `«Tabla»Controller.cs`. Debe **terminar en Controller**: ASP.NET quita esa palabra para formar la URL (`ProductosController` → `/api/Productos`).

Visual Studio genera esta estructura base, sobre la que se trabaja:

Controllers/«Tabla»Controller.cs (generado)

```
using Microsoft.AspNetCore.Mvc;

namespace «Proyecto».Controllers
{
    [Route("api/[controller]")]   // URL base: /api/«Tabla»
    [ApiController]               // valida el JSON automáticamente y responde 400 si viene mal
    public class «Tabla»Controller : ControllerBase
    {
    }
}
```

> **Buena práctica:** también existe **«Controlador de API con acciones que usan Entity Framework»**: eliges la clase modelo y el AppDbContext, y Visual Studio escribe el CRUD completo por ti (y puede instalar un paquete de generación de código). Es útil para avanzar rápido, pero conviene entender primero el código manual de la sección 9.4.

9.3 El campo private readonly y el constructor

El campo y el constructor van **dentro de la clase**, justo después de la llave `{` que abre `public class «Tabla»Controller : ControllerBase`, y **antes** de los métodos GET/POST/PUT/DELETE de la sección 9.4:

Controllers/«Tabla»Controller.cs — dónde va cada parte

```
using Microsoft.AspNetCore.Mvc;
using «Proyecto».Context;                       // ← agregar: para AppDbContext

namespace «Proyecto».Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class «Tabla»Controller : ControllerBase
    {
        // ▼▼▼ AQUÍ: campo + constructor (9.3) ▼▼▼
        private readonly AppDbContext _context;          // (1) campo

        public «Tabla»Controller(AppDbContext context)    // (2) constructor
        {
            _context = context;                           // (3) guardar lo que entrega ASP.NET
        }
        // ▲▲▲ fin de 9.3 ▲▲▲

        // ▼▼▼ AQUÍ: los 5 métodos CRUD (9.4) ▼▼▼
    }
}
```

- El nombre del constructor debe ser **idéntico** al de la clase (`«Tabla»Controller`), y no lleva tipo de retorno.
- El `using «Proyecto».Context;` se agrega a mano (o con **Ctrl+.** sobre `AppDbContext` subrayado en rojo).

Parte	Explicación
`private`	Solo se puede usar dentro de este controlador.
`readonly`	Solo se puede asignar **en la declaración o en el constructor**. Después nadie puede reemplazarlo por error (por ejemplo, `_context = null;` marca error de compilación). Garantiza que el controlador use siempre el mismo contexto durante la petición.
`AppDbContext _context`	Tipo y nombre del campo. El guion bajo es una convención para campos privados (en P04R01 se llama `context`, también es válido).
Constructor con parámetro `AppDbContext context`	**Inyección de dependencias:** tú no escribes `new AppDbContext(...)`. Como en Program.cs se registró `AddDbContext` (Paso 6), ASP.NET crea el contexto con la conexión correcta y lo pasa aquí en cada petición.

9.4 Los métodos CRUD: plantilla completa

Cada método es un **endpoint**. El atributo `[HttpGet]`, `[HttpPost]`, etc. indica el verbo HTTP, y el texto entre paréntesis completa la URL.

Controllers/«Tabla»Controller.cs — plantilla CRUD completa

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;     // ToListAsync, FindAsync, SaveChangesAsync
using «Proyecto».Context;                // AppDbContext
using «Proyecto».Models;                 // «Tabla»

namespace «Proyecto».Controllers
{
    [Route("api/[controller]")]
    [ApiController]
    public class «Tabla»Controller : ControllerBase
    {
        private readonly AppDbContext _context;

        public «Tabla»Controller(AppDbContext context)
        {
            _context = context;
        }

        // GET: api/«Tabla»            -> lista completa
        [HttpGet]
        public async Task<ActionResult<IEnumerable<«Tabla»>>> GetTodos()
        {
            return await _context.«Tabla».AsNoTracking().ToListAsync();
        }

        // GET: api/«Tabla»/5          -> un registro por su llave
        [HttpGet("{id}")]
        public async Task<ActionResult<«Tabla»>> GetPorId(int id)
        {
            var entidad = await _context.«Tabla».FindAsync(id);
            if (entidad == null) return NotFound();            // 404
            return entidad;                                    // 200 + JSON
        }

        // POST: api/«Tabla»           -> crear
        [HttpPost]
        public async Task<ActionResult<«Tabla»>> Crear(«Tabla» entidad)
        {
            _context.«Tabla».Add(entidad);
            await _context.SaveChangesAsync();                 // INSERT; llena la llave generada
            return CreatedAtAction(nameof(GetPorId),           // 201 + ubicación del nuevo registro
                new { id = entidad.«ClavePrimaria» }, entidad);
        }

        // PUT: api/«Tabla»/5          -> actualizar
        [HttpPut("{id}")]
        public async Task<IActionResult> Actualizar(int id, «Tabla» datos)
        {
            if (id != datos.«ClavePrimaria»)
                return BadRequest("El id de la URL no coincide con el del cuerpo.");   // 400

            var entidad = await _context.«Tabla».FindAsync(id);
```

```
            if (entidad == null) return NotFound();            // 404

            _context.Entry(entidad).CurrentValues.SetValues(datos);   // copia los valores nuevos
            await _context.SaveChangesAsync();                 // UPDATE
            return NoContent();                                // 204
        }

        // DELETE: api/«Tabla»/5       -> eliminar
        [HttpDelete("{id}")]
        public async Task<IActionResult> Eliminar(int id)
        {
            var entidad = await _context.«Tabla».FindAsync(id);
            if (entidad == null) return NotFound();            // 404

            _context.«Tabla».Remove(entidad);
            await _context.SaveChangesAsync();                 // DELETE
            return NoContent();                                // 204
        }
    }
}
```

Atención: en `[HttpGet("{id}")]`, las llaves `{ }` sí van tal cual: son la sintaxis de ASP.NET para un **parámetro de ruta**. El nombre (`id`) debe coincidir con el parámetro del método (`int id`). No las confundas con los marcadores «» de este manual.

Error común: tres errores frecuentes al copiar la plantilla CRUD: **(a) tipo de retorno de GetPorId.** `GetTodos` devuelve una **lista** y `GetPorId` devuelve **un solo registro**. Si copias la firma de `GetTodos` en `GetPorId`, `return entidad;` marca el error **CS0029** (no se puede convertir «Tabla» en `ActionResult<IEnumerable<«Tabla»>>`). **(b) El nombre después de `_context.` es el del DbSet**, con las mismas mayúsculas: si en AppDbContext escribiste `DbSet<Rutas> rutas`, en el controlador es `_context.rutas`. Recomendación: nombra el DbSet igual que la tabla y la clase (`DbSet<Rutas> Rutas`). **(c)** `using` **que Visual Studio agrega solo por error** al aceptar una sugerencia con Tab o Enter (por ejemplo `using static System.Runtime.InteropServices.JavaScript.JSType;`, `using System.Drawing;`, `using Microsoft.Identity.Client;`) o uno repetido. Bórralos: aparecen en gris (sin usar). **Clic derecho → Quitar y ordenar usings** (Ctrl+R, Ctrl+G) limpia todos los que sobran.

(a) Firmas correctas

```
public async Task<ActionResult<IEnumerable<«Tabla»>>> GetTodos()   // lista
public async Task<ActionResult<«Tabla»>> GetPorId(int id)          // uno solo (sin IEnumerable)
```

9.5 Rutas que quedan y qué responde cada una

Verbo	URL	Método	Cuerpo (JSON)	Respuestas
GET	`/api/«Tabla»`	GetTodos	—	200 lista
GET	`/api/«Tabla»/5`	GetPorId	—	200 registro · 404 no existe
POST	`/api/«Tabla»`	Crear	el objeto nuevo	201 creado · 400 datos inválidos
PUT	`/api/«Tabla»/5`	Actualizar	el objeto completo	204 actualizado · 400 · 404
DELETE	`/api/«Tabla»/5`	Eliminar	—	204 eliminado · 404

Para recordarlo: por cada tabla, el controlador tiene siempre **5 métodos**: 2 GET (todos y por id), 1 POST, 1 PUT y 1 DELETE. Cambian solo tres palabras: «Tabla», «ClavePrimaria» y el nombre del DbSet. Los que reciben un `id` siguen el mismo patrón de 4 pasos: **1. Buscar:** `FindAsync(id)` · **2. Si no existe:** `return NotFound();` (404) · **3. Hacer el cambio:** `Add` / `SetValues` / `Remove`, y después `SaveChangesAsync()` · **4. Responder:** registro (200), `CreatedAtAction` (201) o `NoContent()` (204).

Código	Significado	Método de C#
200 OK	Todo bien, con datos.	`Ok(datos)` o `return datos;`
201 Created	Se creó un registro.	`CreatedAtAction(...)`
204 No Content	Se hizo, sin datos que devolver.	`NoContent()`
400 Bad Request	El cliente mandó algo inválido.	`BadRequest(...)` (o automático con [ApiController])
404 Not Found	No existe el registro.	`NotFound()`
409 Conflict	Choca con datos existentes (duplicado).	`Conflict(...)`
500 Internal Server Error	Falló el servidor o la BD.	Excepción no controlada

9.6 Por qué async/await y no la versión síncrona

`ToListAsync`, `FindAsync` y `SaveChangesAsync` liberan el hilo del servidor mientras la BD responde, así la API atiende más peticiones a la vez. La versión síncrona (`ToList()`, `Find()`, `SaveChanges()`, como en P04R01) funciona, pero escala peor. Cuando un método usa `await`, debe declararse `async Task<...>`.

Error común: evita envolver todo en `try { ... } catch (Exception ex) { return BadRequest(ex.Message); }`: convierte errores del servidor (500) en errores del cliente (400) y expone detalles internos. Deja que las excepciones inesperadas respondan 500 y atrapa solo las que esperas (por ejemplo, `DbUpdateException` por un duplicado → `Conflict`).

10. Paso 8 — launchSettings.json: cómo se ejecuta la API

10.1 Dónde está y para qué sirve

En `Properties/launchSettings.json` (carpeta Properties del proyecto). Define **qué pasa al presionar Iniciar en Visual Studio** (o `dotnet run`): en qué puertos escucha la API, si se abre el navegador y en qué página, y en qué entorno corre. **No es obligatorio modificarlo**; lo importante es **entenderlo**, porque de aquí sale la URL que usará el frontend.

Properties/launchSettings.json (resumido; los números de puerto los genera la plantilla al azar)

```
{
  "profiles": {
    "http": {                                          <- nombre del perfil (aparece junto al botón Iniciar)
      "commandName": "Project",
      "launchBrowser": true,                           <- abre el navegador al iniciar
      "launchUrl": "swagger",                          <- página que abre (swagger, api/«Tabla», ...)
      "applicationUrl": "http://localhost:5283",       <- PUERTO(S) de la API
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"        <- entorno: activa Swagger y appsettings.Development.json
      }
    },
    "https": { ... "applicationUrl": "https://localhost:7202;http://localhost:5283" ... },
    "IIS Express": { ... }
  }
}
```

10.2 Qué revisar o cambiar

Necesidad	Qué hacer
Saber la URL que usará el frontend	Mira `applicationUrl` del perfil que ejecutas. Esa URL va en el frontend (en React con Vite, en un archivo `.env`; en Angular, en `environment.ts`).
Que se abra Swagger al iniciar	`"launchUrl": "swagger"` en el perfil que usas. En .NET 9/10 con Scalar: `"scalar/v1"`.
Elegir el perfil	Lista desplegable junto al botón verde Iniciar de Visual Studio. En terminal: `dotnet run --launch-profile http`.
Evitar problemas de certificado y CORS en desarrollo	Usa el perfil **http**. Con **https**, la API redirige las peticiones http a https; algunos navegadores bloquean esa redirección o el certificado de desarrollo.
Cambiar el puerto	Edita el número en `applicationUrl` y actualiza la URL en el frontend.

> **Error común:** en `applicationUrl`, varias URLs se separan con `;` **sin espacios** (`"http://localhost:5283;http://localhost:7374"`). Un espacio después del `;` hace que la API se cierre al arrancar con «Unrecognized scheme in server address». Tampoco pongas comentarios `//`: Visual Studio los tolera, pero `dotnet run` no puede leer el archivo.

> **Nota:** cada perfil abre su propia página. Si «ya no se abre Swagger», revisa el `launchUrl` del perfil seleccionado: en P04R01 el perfil http abre `api/usuarios` y los perfiles https e IIS Express abren `swagger`.

11. Paso 9 — Ejecutar y probar la API

Una API no está terminada hasta probar cada endpoint.

11.1 Ejecutar

Cómo	Qué hace
F5 (Iniciar depuración)	Ejecuta con depurador: puedes poner puntos de interrupción (clic en el margen izquierdo del código).
Ctrl+F5 (Iniciar sin depurar)	Ejecuta más rápido, sin depurador.
`dotnet run` (terminal)	Igual, desde la línea de comandos.

Se abre una consola con `Now listening on: http://localhost:«puerto»` y el navegador en la página definida por `launchUrl`.

11.2 Probar

Herramienta	Cómo
Swagger UI	Abre `http://localhost:«puerto»/swagger` → despliega un endpoint → **Try it out** → llena los datos → **Execute**. Muestra la respuesta y el código.
Navegador	Solo para GET: abre `http://localhost:«puerto»/api/«Tabla»`.
Archivo `.http` de Visual Studio	Escribe la petición y presiona «Send request» encima de ella (ejemplo abajo).
Postman	Nueva petición → verbo + URL → pestaña Body → raw → JSON.

«Proyecto».http

```
@host = http://localhost:5283

GET {{host}}/api/«Tabla»
Accept: application/json

###

POST {{host}}/api/«Tabla»
Content-Type: application/json

{
  "nombre": "Ejemplo",
  "activo": true
}
```

11.3 Solución de problemas frecuentes

Síntoma / mensaje	Causa	Solución
«A network-related or instance-specific error…»	Nombre de servidor o instancia incorrecto, o el servicio SQL apagado.	Copia el nombre exacto de SSMS (ej. `PC\\SQLEXPRESS`); revisa el servicio en services.msc.
«The certificate chain was issued by an authority that is not trusted»	SQL Server sin certificado de confianza (EF Core 7+).	Agrega `TrustServerCertificate=True;` a la cadena.
«Login failed for user…» / «Cannot open database»	Sin permisos o la BD no existe / nombre mal escrito.	Verifica el usuario y el nombre de la BD en SSMS.
«Invalid object name '«X»'»	La tabla se llama distinto a la propiedad DbSet.	Usa `[Table("nombre_real")]` en el modelo o `ToTable` en OnModelCreating.
«The entity type '«X»' requires a primary key»	La llave no se llama `Id` ni `«Clase»Id` y no tiene `[Key]`.	Agrega `[Key]` a la propiedad llave.
«Unable to resolve service for type AppDbContext»	Falta `AddDbContext` en Program.cs.	Paso 6, sección 8.2.
En el navegador: «blocked by CORS policy»	Origen no permitido, `UseCors` faltante o mal ubicado, o falta `AllowAnyMethod`.	Sección 8.3. El origen debe ser **exacto** (protocolo + puerto, sin `/` final).
El frontend dice «Failed to fetch» / no conecta	La API no corre, o corre en otro puerto/perfil.	Compara `applicationUrl` (Paso 8) con la URL del frontend.
400 con «The X field is required»	`[ApiController]` valida: una propiedad `string` (sin `?`) no vino en el JSON.	Envíala o declárala `string?` si acepta NULL.
415 Unsupported Media Type	La petición no indica que manda JSON.	Encabezado `Content-Type: application/json`.
«A possible object cycle was detected»	Relaciones de navegación en ambos sentidos.	DTOs o `ReferenceHandler.IgnoreCycles` (sección 5.6).
Error NU1202 / paquete incompatible	Versión de EF Core distinta a la de .NET.	Instala la versión con el mismo número mayor (Paso 2).

12. ¿Qué cambia si cambio de base de datos, frontend, versión o herramienta?

12.1 Cambiar de base de datos

Paso	¿Cambia?	Qué cambia
Paso 2 — NuGet	**Sí**	Instalas el proveedor de tu BD (tabla 4.4) en vez de `...SqlServer`.
Paso 3 — Modelos	Casi no	Los tipos de dato de la tabla 5.4 son de SQL Server; cada motor tiene equivalentes (ej. `SERIAL` → `int` en PostgreSQL, `BOOLEAN` → `bool`).
Paso 4 — DbContext	No	—
Paso 5 — Conexión	**Sí**	Formato de la cadena (tabla 7.3).
Paso 6 — Program.cs	**Sí, una palabra**	`UseSqlServer` → `UseNpgsql`, `UseMySql`, `UseSqlite`, `UseOracle`.
Pasos 7, 8 y 9	No	Los controladores no saben qué BD hay detrás: esa es la ventaja de EF Core.

> **Nota:** bases **no relacionales** (MongoDB, Redis, etc.) normalmente se usan con su propio driver (por ejemplo `MongoDB.Driver`) en lugar de EF Core; los Pasos 2 a 6 cambian. Los Pasos 1, 7, 8 y 9 se mantienen.

12.2 Cambiar de frontend

Solo cambia el origen en CORS (sección 8.3). La API **no sabe ni le importa** qué frontend la consume.

12.3 Cambiar de versión de .NET

Tema	.NET 8	.NET 9	.NET 10
Visual Studio	2022 (17.8+)	2022 (17.12+)	**2026** (18.0+)
Paquetes EF Core	8.0.x	9.0.x	10.0.x
Swagger en la plantilla	Swashbuckle incluido (`/swagger`)	Solo OpenAPI JSON; UI opcional	Solo OpenAPI JSON (3.1); UI opcional
Pasos 2 a 9	Iguales	Iguales (excepto Swagger, 8.5)	Iguales (excepto Swagger, 8.5)

12.4 Usar VS Code o la terminal en lugar de Visual Studio

Acción	Visual Studio	Terminal / VS Code
Crear proyecto	Plantilla «ASP.NET Core Web API»	`dotnet new webapi --use-controllers -n «Proyecto»`
Agregar paquete	Administrar paquetes NuGet	`dotnet add package «Paquete» --version «x.y.*»`

Acción	Visual Studio	Terminal / VS Code
Compilar	`Ctrl+Shift+B`	`dotnet build`
Ejecutar	F5 / Ctrl+F5	`dotnet run --launch-profile http`
Generar modelos desde la BD	`Scaffold-DbContext` (PM>)	`dotnet ef dbcontext scaffold ...`
Migraciones	`Add-Migration` / `Update-Database`	`dotnet ef migrations add` / `dotnet ef database update`
Crear carpeta / clase	Clic derecho → Agregar	Crear el archivo `.cs` en la carpeta; el namespace debe coincidir con la ruta.

12.5 El mismo modelo mental en otros lenguajes

Si algún día haces una API en otra tecnología, las piezas son las mismas con otro nombre:

Pieza	ASP.NET Core (este manual)	Node.js (Express)	Java (Spring Boot)	Python (FastAPI)
Modelo / entidad	Clase en `Models/`	Modelo de Prisma / Sequelize	Clase `@Entity`	Modelo de SQLAlchemy
Acceso a datos (ORM)	DbContext + DbSet (EF Core)	Prisma Client / Sequelize	JPA Repository	Session de SQLAlchemy
Cadena de conexión	appsettings.json	`.env`	application.properties	`.env`
Registro de servicios	Program.cs	app.js / index.js	Anotaciones / @Configuration	main.py
Endpoints	Controllers	Routers	@RestController	Routers / @app.get
CORS	AddCors / UseCors	paquete `cors`	@CrossOrigin / WebMvcConfigurer	CORSMiddleware

13. Observaciones sobre el proyecto P04R01_APIREST

Observación	Recomendación
`<TargetFramework>net8.0</TargetFramework>` con EF Core **6.0.0**.	Actualizar ambos paquetes de EF Core a 8.0.x (o migrar todo a .NET 10).
Siguen `WeatherForecastController.cs` y `WeatherForecast.cs` de la plantilla.	Borrarlos: no forman parte de la API.
Los métodos del controlador usan `try/catch` con `BadRequest(ex.Message)` para todo.	Ver la nota de la sección 9.6.
Los métodos son síncronos (`ToList`, `Find`, `SaveChanges`).	Usar la versión async (sección 9.6).
`POST` responde `Ok()` sin datos.	Responder `CreatedAtAction(...)` con el registro creado (201).
Ya corregido: CORS sin `AllowAnyMethod`, espacio en `applicationUrl`, cadena con opciones contradictorias.	Ver `HISTORIAL.md` del proyecto.

14. Buenas prácticas y siguientes mejoras

Con los Pasos 0 a 9 la API funciona. Para una API profesional, estas son las mejoras más comunes, en el orden sugerido:

Mejora	Qué es	Para qué
DTOs (Data Transfer Objects)	Clases separadas para lo que entra y sale de la API (ej. `ProductoCrearDto`), distintas de las entidades.	No exponer columnas internas, evitar ciclos JSON y controlar qué puede modificar el cliente.
Validación	Atributos `[Required]`, `[EmailAddress]`, `[Range]`... en los DTOs.	Rechazar datos inválidos con 400 antes de tocar la BD.
Capa de servicios	Clases en `Services/` con la lógica de negocio; el controlador solo recibe y responde.	Controladores cortos, lógica reutilizable y fácil de probar.
Manejo global de errores	`builder.Services.AddProblemDetails();` + `app.UseExceptionHandler();`	Todas las fallas responden con un formato JSON estándar, sin `try/catch` en cada método.
Paginación y filtros	Parámetros `?pagina=1&tamano=20` con `Skip`/`Take`.	No devolver miles de registros de golpe.
Autenticación / autorización	JWT Bearer + `[Authorize]` en los controladores.	Que solo usuarios válidos usen ciertos endpoints.
Logging	`ILogger<T>` inyectado en el constructor.	Registrar errores y eventos para diagnosticar problemas.
Pruebas automáticas	Proyecto xUnit que prueba los servicios y endpoints.	Detectar errores antes de publicar.

15. Lista de verificación final

Antes de dar por terminada una API, confirma cada punto:

- Proyecto creado con **ASP.NET Core Web API** y **Use controllers** marcado.
- Paquetes de EF Core instalados, con la **misma versión mayor** que .NET.
- Una clase en `Models/` **por cada tabla**, con una propiedad por columna, tipos correctos, `?` en las que aceptan NULL y `[Key]` si hace falta.
- `Context/AppDbContext.cs` **hereda de DbContext**, tiene el constructor con `DbContextOptions<AppDbContext>` y **un DbSet por tabla**.
- `appsettings.json` tiene `ConnectionStrings` → «NombreConexion» con la cadena correcta (y `TrustServerCertificate=True` en SQL Server local).
- `Program.cs` registra `AddDbContext` con el mismo «NombreConexion» y el método `UseXxx` de tu BD.
- `Program.cs` registra `AddCors` con los orígenes exactos del frontend, `AllowAnyHeader()` **y** `AllowAnyMethod()`, y llama `app.UseCors("«Politica»")` antes de `MapControllers`.
- Un controlador por tabla expuesta, que termina en `Controller`, con `[Route("api/[controller]")]`, `[ApiController]`, el campo `private readonly AppDbContext` y los 5 métodos CRUD.
- Sabes qué perfil de `launchSettings.json` usas y cuál es su `applicationUrl`.
- Compila con 0 errores (`Ctrl+Shift+B`).
- Probaste en Swagger (o .http / Postman): GET lista, GET por id (existente y no existente), POST, PUT y DELETE.
- El frontend consume la API sin errores de CORS.

Apéndice A — Ejemplo completo para una base de datos con N tablas

Base de datos de ejemplo `Tienda` con 3 tablas, para ver cómo se repite el patrón:

SQL

```
CREATE TABLE Categorias (IdCategoria INT IDENTITY PRIMARY KEY, Nombre NVARCHAR(80) NOT NULL);
CREATE TABLE Productos  (IdProducto INT IDENTITY PRIMARY KEY, Nombre NVARCHAR(100) NOT NULL,
                         Precio DECIMAL(10,2) NOT NULL, IdCategoria INT NOT NULL
                         REFERENCES Categorias(IdCategoria));
CREATE TABLE Clientes   (IdCliente INT IDENTITY PRIMARY KEY, Nombre NVARCHAR(100) NOT NULL,
                         Email NVARCHAR(100) NULL);
```

Pieza	Categorias	Productos	Clientes
Modelo (Paso 3)	`Models/Categorias.cs`	`Models/Productos.cs`	`Models/Clientes.cs`
DbSet (Paso 4)	`DbSet<Categorias> Categorias`	`DbSet<Productos> Productos`	`DbSet<Clientes> Clientes`
Controlador (Paso 7)	`CategoriasController.cs`	`ProductosController.cs`	`ClientesController.cs`
URL base	`/api/Categorias`	`/api/Productos`	`/api/Clientes`

Context/AppDbContext.cs

```
using Microsoft.EntityFrameworkCore;
using Tienda.Models;

namespace Tienda.Context
{
    public class AppDbContext : DbContext
    {
        public AppDbContext(DbContextOptions<AppDbContext> options) : base(options) { }

        public DbSet<Categorias> Categorias { get; set; }
        public DbSet<Productos> Productos { get; set; }
        public DbSet<Clientes> Clientes { get; set; }
    }
}
```

Models/Productos.cs

```
using System.ComponentModel.DataAnnotations;
using System.ComponentModel.DataAnnotations.Schema;

namespace Tienda.Models
{
    public class Productos
    {
        [Key]
        public int IdProducto { get; set; }          // [Key] necesario: no se llama Id ni ProductosId

        [Required, StringLength(100)]
        public string Nombre { get; set; } = string.Empty;

        [Column(TypeName = "decimal(10,2)")]
        public decimal Precio { get; set; }

        public int IdCategoria { get; set; }         // FK
        [ForeignKey(nameof(IdCategoria))]
        public Categorias? Categoria { get; set; }   // navegación
    }
}
```

`CategoriasController`, `ProductosController` y `ClientesController` son la plantilla de la sección 9.4, reemplazando «Tabla» por el nombre de la tabla y «ClavePrimaria» por `IdCategoria`, `IdProducto` e `IdCliente`.

Apéndice B — Comandos de referencia

Para	Consola del Administrador de paquetes (PM>)	Terminal (dotnet CLI)
Ver SDK instalados	—	`dotnet --list-sdks`
Crear proyecto	— (usar la plantilla)	`dotnet new webapi --use-controllers -n «Proyecto»`
Instalar paquete	`Install-Package «Paquete» -Version «x.y.*»`	`dotnet add package «Paquete» --version «x.y.*»`
Instalar herramienta EF (una vez)	— (viene con el paquete Tools)	`dotnet tool install --global dotnet-ef`
Generar modelos desde la BD	`Scaffold-DbContext "Name=ConnectionStrings:«NombreConexion»" «Proveedor» -OutputDir Models -ContextDir Context -Context AppDbContext -DataAnnotations`	`dotnet ef dbcontext scaffold "Name=ConnectionStrings:«NombreConexion»" «Proveedor» -o Models --context-dir Context -c AppDbContext --data-annotations`
Crear migración	`Add-Migration «Nombre»`	`dotnet ef migrations add «Nombre»`
Aplicar migraciones	`Update-Database`	`dotnet ef database update`
Compilar	Menú Compilar (`Ctrl+Shift+B`)	`dotnet build`
Ejecutar con un perfil	Lista junto al botón Iniciar	`dotnet run --launch-profile http`